Assignment 2

Introduction to UNIX Lab (NCSC102)

Instructions:

- **Once completed, show your code to the respective TAs.**
- **You may ask the TAs for help if you're encountering syntax errors or having trouble understanding the questions.**

Lab Objectives

By the end of this lab, you will be able to:

- Navigate a file efficiently using Vim's motions without using the mouse.
- Perform fundamental editing operations like deleting, changing, copying, and pasting text.
- Understand and utilize Vim's core modes: Normal, Insert, and Visual.
- Use the command-line mode for powerful actions like search-and-replace and saving files.
- Apply operators on visually selected blocks of text for bulk modifications.
- Run external shell commands from within Vim.

Part 1: The Absolute Basics

Task 1.1: Setup

1. Open your terminal.
2. Create a new file named `vim_practice_1.txt` by typing `vim vim_practice_1.txt` and pressing Enter.
3. You are now in **Normal Mode**. This is the default mode for navigation and commands.

Task 1.2: Entering Insert Mode & Basic Editing

1. Press the **i** key to switch to **Insert Mode**. Notice the **-- INSERT --** text at the bottom of the screen.
2. Copy and paste the following paragraph into the file:

   Vim is a text editor that is modal, unlike most modern editors. This menas it has different modes for inserting text and for navigating or manipulating it. The separation of modes keeps fingers on the home row and makes editing faster once mastered.
3. Press the **Esc** key to return to **Normal Mode**.
4. The paragraph has two typos: "menas" should be "means" and "faster" is missing a space before it.
   - Use the **h**, **j**, **k**, **l** keys to move your cursor to the 'n' in "menas".
   - Press **x** to delete the 'n'. Now, move the cursor to the space after 'a' and press **i** to enter Insert mode. Type **n** and press **Esc**.
   - Navigate to the 'f' in "faster". Press **i**, type a space, and press **Esc**.

Task 1.3: More Ways to Insert

1. Navigate to the end of the line. Press **A** to **append** text at the very end of the line and enter Insert Mode.
2. Add the text: " It is very powerful."
3. Press **Esc**.
4. Navigate to the middle of the last sentence. Press **o** to open a new line **below** the current one and enter Insert Mode.
5. Type: "This is a new line."
6. Press **Esc**.

Task 1.4: Saving and Quitting

1. To save your work, enter the command **:w** and press Enter.
2. Now, quit the editor with the command **:q**.
3. Re-open the file to see your saved changes.
4. A shortcut to save and quit is **:wq**. Try it now.

Part 2: The Vim Grammar - Operators & Motions

Here you'll learn Vim's core strength: combining operators (verbs) with motions (nouns).

Task 2.1: Setup

1. Create a new file **`code.c`**.
2. Enter insert mode and paste the following poorly formatted C code:

```c
// This is a comment line that is way too long and should probably be
   removed.
#include <stdio.h>

int calculate_sum(int a,int b)
{
// Another unnecessary comment.
    int result = a + b; // sum the two numbers
    printf("The final result is: %d\n", result); // print the result
    return result;
}
// The function ends here.
```

Task 2.2: Deletions

1. You are in Normal Mode. Navigate to the first line. To delete the entire line, press **`dd`**.
2. Go to the line that says **`// Another unnecessary comment.`** and delete it with **`dd`**.
3. Go to the last line and delete it with **`dd`**.
4. On the **`printf`** statement line, move your cursor to the first **`/`**. You want to delete from the cursor to the end of the line. Type **`d$`**.
5. On the **`int result =`** line, move your cursor to the first **`/`**. Do the same thing: **`d$`**.

Task 2.3: Changing Text

1. In the function signature, the variable **`b`** is not very descriptive. Let's change it to **`number_two`**.
2. Move your cursor to the **`b`**. Type **`cw`** (change word) and then type **`number_two`**. Press **`Esc`**.
3. Now fix the line below. Move to **`b`** and use the **`.`** (dot) command to **repeat the last change**.

Task 2.4: Yanking and Putting (Copy-Paste)

1. Let's add another variable. Go to the **`int result =`** line.
2. Yank (copy) the entire line by typing **`yy`**.
3. Press **`p`** to put (paste) the line below the current one.

4. On the new line, use **cw** to change **result** to **another_result** and **a + number_two** to **a * 10**.

5. Your code should now look like this:

```
#include <stdio.h>

int calculate_sum(int a, int number_two)
{
    int result = a + number_two;
    int another_result = a * 10;
    printf("The final result is: %d\n", result);
    return result;
}
```

6. Save and quit the file (**:wq**).

Part 3: Search and Replace

Task 3.1: Searching

1. Create a file **story.txt** and paste the following text:

   The quick brown fox jumps over the lazy dog. The fox is a clever fox, and that fox knows it. The dog, however, is not as clever.

2. From the top of the file, search for the word "fox". Type **/fox** and press Enter.

3. Press **n** to jump to the next match and **N** to jump to the previous one. Do this a few times.

4. Move your cursor over the word "dog". Press ***** to search for the word under the cursor.

Task 3.2: Substitution

1. The story is boring. Let's replace the "fox" with "cat".

2. Enter the command **:%s/fox/cat/g**. Let's break it down:

   - **:** enters command mode.
   - **%** means "on every line in the file" (shorthand for **1,$**).
   - **s** is for substitute.
   - **/fox/cat/** is the pattern and replacement.
   - **g** means "global", so it replaces every instance on a line, not just the first.

3. Press **u** to undo the change.

4. Let's try again, but this time with confirmation. Type **:%s/fox/cat/gc** and press Enter.

5. For each match, Vim will ask you what to do. Press **y** (yes) for the first two and **n** (no) for the last two.

6. Save and quit.

Part 4: Visual Mode & External Commands

Task 4.1: Visual Indentation

1. Create a new file, **indent.c**, and paste the following unindented code:

```c
#include <stdio.h>

int main() {
int x = 10;
int y = 20;
int sum = x + y;
printf("The sum is %d\n", sum);
return 0;
}
```

2. The five lines inside the **main** function's curly braces need to be indented.

3. Move your cursor to the **i** in the first **int**.

4. Press **Ctrl-v** or **ctrl-q** in Windows to enter **Visual Block Mode**.

5. Press **j** four times to select the block of lines down to the **return** statement.

6. Press **Shift-i** (capital I).

7. Type four spaces (a standard C indent).

8. Press **Esc**. The indentation will be applied to all selected lines.

Task 4.2: Running Shell Commands

1. Go to the very end of the file (**G**).

2. Press **o** to open a new line.

3. Let's insert the current date and time into the file as a comment. Type the command **:r !date** and press Enter. This reads the output of the **date** shell command into your file.

4. Save and quit the file.

Bonus Challenge (If you finish early)

1. **Named Registers:** In **`story.txt`**, yank the first sentence into register **`a`** (**`"ayy`**). Delete the second sentence. Go to the end of the file and paste the sentence from register **`a`** (**`"ap`**).

2. **Advanced Substitution:** In **`code.c`**, use a substitution command to comment out the function body (lines 6 through 8) using C's preprocessor. (Hint: **`:6,8s/^/# /`**). While **`/* ... */`** is more common for block comments, this method is excellent for practicing line-based substitution.

3. **Help System:** Use the built-in help to learn about a new command. For example, type **`:help substitute`** to read the full documentation for the substitute command.

4. How do I add the letter 'A' in front of each line of text I've selected using Visual Block mode in Vim?